# Fused3S: Fused Sparse-Dense Graph Attention on Tensor Cores

Mayank Choudhary
mayank.choudhary@iitgn.ac.in
Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

## Abstract

Graph transformers rely on sparse attention to capture structural relationships, but the standard three-kernel pipeline—Sampled Dense-Dense Matrix Multiply (SDDMM), Softmax, and Sparse Matrix-Matrix Multiply (SpMM)—incurs significant memory traffic from materializing intermediate results between stages. We present **Fused3S**, a high-performance CUDA kernel that fuses all three stages into a single GPU kernel operating on a block-sparse representation of the graph adjacency. The kernel leverages NVIDIA Tensor Cores via the WMMA API for 16×16×16 FP16 matrix multiplications, and employs an online softmax algorithm for numerically correct normalization without a separate pass. We develop three progressive kernel versions: `v0` with per-block softmax, `v1` with globally correct online softmax, and `v2` with register-resident output accumulators that eliminate shared memory round-trips. On the Cora and Citeseer citation graphs using an RTX 4050 (sm_89), Nsight Compute profiling shows `v2` consistently achieves the fastest kernel execution (e.g., 8.77 $\mu$s on Cora, 9.31 $\mu$s on Citeseer), yielding up to 1.53× speedup over `v0` and 2.46× over `v1`. Roofline analysis confirms that `v2` attains the highest hardware utilization among all versions. All kernel versions are verified against CPU ground truth with element-wise tolerance of 0.05.

## 1 Introduction

Graph Neural Networks (GNNs) have emerged as the dominant paradigm for learning on graph-structured data, with applications spanning citation networks [4], social networks, molecular property prediction, and recommendation systems. The introduction of attention mechanisms to graphs via Graph Attention Networks (GATs) [10] enabled nodes to learn adaptive, data-dependent aggregation weights over their neighborhoods, significantly improving representational power over fixed-weight methods like GCN [4].

The computation underlying sparse graph attention can be decomposed into three stages:

(1) **SDDMM** (Sampled Dense-Dense Matrix Multiply): Compute attention scores $S_{ij} = Q_i \cdot K_j^T$ only for edges $(i, j)$ present in the graph.
(2) **Softmax**: Normalize scores row-wise to produce attention weights $P_{ij} = \text{softmax}_j(S_{ij})$.
(3) **SpMM** (Sparse Matrix-Matrix Multiply): Aggregate neighbor features weighted by attention: $O_i = \sum_j P_{ij} V_j$.

In standard implementations, each stage is executed as a separate GPU kernel, requiring the intermediate tensors $S$ and $P$ to be written to and read from global memory between stages. This *memory-bound* pattern is particularly wasteful for sparse graphs where the computation-to-memory ratio is already low.

Recent work by Li and Chandramowlishwaran [5] proposed fusing all three stages into a single kernel, dramatically reducing memory traffic by keeping intermediate values in shared memory and registers. Concurrently, FlashAttention [1] demonstrated the effectiveness of fused attention kernels for dense transformers, using online softmax [6] to avoid materializing the full attention matrix.

In this project, we implement the Fused3S approach from scratch in CUDA, targeting NVIDIA Tensor Cores via the WMMA API. We develop three progressive kernel versions that systematically explore the design space of fused sparse attention:

- **`v0`**: A baseline fused kernel with per-block softmax and 1D thread layout.
- **`v1`**: Introduces online softmax for globally correct attention weights with a 2D thread layout.
- **`v2`**: Moves the output accumulator to WMMA register fragments, eliminating shared memory round-trips and achieving the fastest kernel execution time while maintaining correctness.

We evaluate all versions on the Cora [8] and Citeseer citation graphs using an RTX 4050 GPU (sm_89) and verify correctness against a CPU ground truth implementation.

## 2 Background

### 2.1 Sparse Graph Attention

Given a graph $G = (V, E)$ with $N = |V|$ nodes and adjacency defined by edge set $E$, sparse graph attention computes:

$$S_{ij} = Q_i \cdot K_j^T, \quad \forall (i, j) \in E, \tag{1}$$

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_{k:(i,k)\in E} \exp(S_{ik})}, \tag{2}$$

$$O_i = \sum_{j:(i,j)\in E} P_{ij} \cdot V_j, \tag{3}$$

where $Q, K, V \in \mathbb{R}^{N\times d}$ are the query, key, and value matrices with feature dimension $d$. Equations 1–3 correspond to SDDMM, Softmax, and SpMM respectively.

### 2.2 NVIDIA Tensor Cores and WMMA

NVIDIA Tensor Cores are specialized hardware units that perform matrix multiply-accumulate (MMA) operations on small tiles. The Warp Matrix Multiply-Accumulate (WMMA) API [7] exposes these units through C++ fragment types. For our implementation, we use 16×16×16 tiles:

$$D_{16\times16} = A_{16\times16} \cdot B_{16\times16} + C_{16\times16}, \tag{4}$$

where $A, B$ are FP16 inputs and $C, D$ are FP32 accumulators. A single `wmma::mma_sync` call computes this operation cooperatively across all 32 threads in a warp.

## 2.3 Online Softmax

The standard softmax requires two passes: one to find the row maximum and one to compute exponentials and normalize. The online softmax algorithm [6] maintains running statistics $(m_{\text{prev}}, \ell_{\text{prev}})$ and updates them as new blocks arrive:

$$m_{\text{new}} = \max(m_{\text{prev}}, m_{\text{block}}), \tag{5}$$

$$\ell_{\text{new}} = \ell_{\text{prev}} \cdot e^{m_{\text{prev}} - m_{\text{new}}} + \sum_j e^{S_j - m_{\text{new}}}, \tag{6}$$

$$O_{\text{new}} = O_{\text{prev}} \cdot e^{m_{\text{prev}} - m_{\text{new}}} + P_{\text{block}} \cdot V_{\text{block}}. \tag{7}$$

This enables correct softmax computation in a single streaming pass over column blocks.

# 3 System Design

## 3.1 CSR to Block-Sparse Conversion

The input graph is stored in Compressed Sparse Row (CSR) format. We convert it to a block-sparse representation aligned with the WMMA tile size of 16×16. The conversion groups rows into *Row Windows* of 16 consecutive nodes each. For each Row Window, we collect all unique 16-column blocks that contain at least one edge from any row in the window:

$$\text{num\_row\_windows} = \lceil N/16 \rceil. \tag{8}$$

The resulting data structure consists of:

- `window_offsets[]`: For each Row Window $w$, stores the start index in `block_col_indices` where its active column blocks begin.
- `block_col_indices[]`: A flat array of active 16×16 column block indices across all Row Windows.

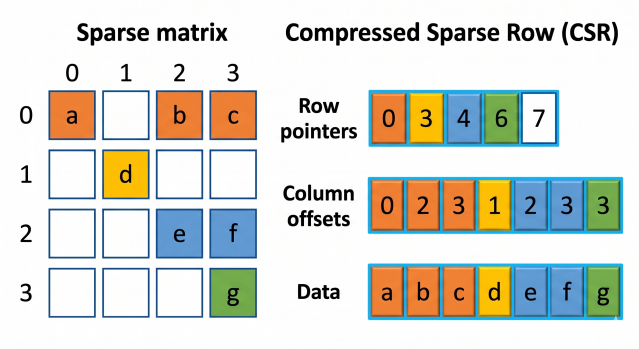


**Figure 1: CSR to block-sparse format conversion. Rows are grouped into 16-row windows, and edges are mapped to 16×16 column blocks. Each Row Window references only its active blocks.**

## 3.2 Load Balancing

Different Row Windows may have vastly different numbers of active column blocks. To ensure balanced GPU utilization, we sort Row Windows in descending order by their active block count. This assigns heavier windows to earlier thread blocks, which are scheduled first by the GPU's block scheduler:

$$\texttt{window\_order} = \text{argsort}_{\text{desc}}(\texttt{window\_offsets}[w+1] - \texttt{window\_offsets}[w]). \tag{9}$$

## 3.3 Fused Kernel Architecture

Each thread block processes one Row Window. The kernel fuses all three stages (SDDMM, Softmax, SpMM) into a single launch, with data flowing through shared memory and registers instead of global memory. The Q tile for the Row Window is loaded once into shared memory and reused across all column blocks.

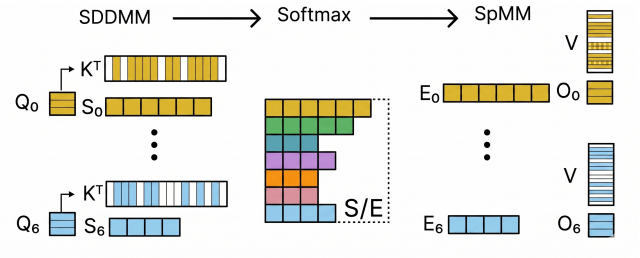


**Figure 2: Fused kernel pipeline for one Row Window. The three stages communicate through shared memory and registers, eliminating global memory round-trips for intermediate tensors $S$ and $P$.**

# 4 Kernel Versions

## 4.1 v0: Per-Block Softmax + 1D Thread Layout

The baseline fused kernel uses a 1D thread layout with 128 threads (4 warps × 32 threads). Column blocks are distributed across warps (`b = block_start + warp_id; b += WARPS_PER_BLOCK`), with each warp independently processing its assigned blocks.

Each warp computes SDDMM via WMMA (4 MMA ops for $d = 64$), applies a *local* softmax over the 16×16 score block, then accumulates SpMM results into per-warp WMMA accumulators (`frag_O[4]` for 4 feature chunks).

**Limitation:** Softmax is computed independently within each 16×16 block rather than across all active column blocks for a given row. This produces incorrect attention weights when a row has neighbors spanning multiple column blocks.

## 4.2 v1: Online Softmax + 2D Thread Layout

Version 1 addresses the softmax correctness issue by introducing the online softmax algorithm. Key changes:

- **Sequential column-block iteration:** All warps process column blocks in the same order (`for b = block_start..block_end`), enabling coherent running-state updates.
- **2D thread layout:** `dim3(32, WARPS_PER_BLOCK)` with `threadIdx.y` as warp ID.
- **Running state:** Shared memory arrays `row_max[16]` and `row_sum[16]` maintain the online softmax state.
- **Output accumulator in shared memory:** `smem_O[16][64]` is rescaled by the correction factor $e^{m_{\text{old}} - m_{\text{new}}}$ each iteration before SpMM accumulation.

**Limitation:** The output rescaling requires reading and writing 1,024 floats (16×64) from shared memory every iteration, plus an

extra `__syncthreads()` after SpMM since all warps write to shared memory.

### 4.3 v2: Register-Resident Output + Fragment Rescaling

Version 2 eliminates the shared memory bottleneck of v1:

- **Register-resident `frag_O`:** Each warp permanently owns one WMMA accumulator fragment for its feature chunk (warp $i \rightarrow$ chunk $i$). The fragment lives in registers for the entire kernel lifetime.
- **In-register rescaling:** Instead of rescaling `smem_O`, we directly multiply `frag_O.x[]` elements by per-row correction factors, using the known sm_80+ WMMA fragment layout:

$$\text{groupID} = \text{lane\_id}/4, \tag{10}$$

$$\text{elements } [0, 1, 4, 5] \rightarrow \text{row} = \text{groupID}, \tag{11}$$

$$\text{elements } [2, 3, 6, 7] \rightarrow \text{row} = \text{groupID} + 8. \tag{12}$$

- **Removed `__syncthreads()`:** After SpMM, since output writes go to registers (not shared memory), the barrier between SpMM and the next iteration's SDDMM is unnecessary.

This achieves the fastest kernel execution time among all versions while maintaining v1's globally correct softmax.

**Table 1: Shared memory layout across kernel versions. v2 eliminates smem_O by using register fragments.**

| Buffer | Size | v0 | v1/v2 |
|---|---|---|---|
| `smem_Q[16][64]` | 2 KB | yes | yes |
| `smem_S[16][16]` | 1 KB | per-warp | shared |
| `smem_P[16][16]` | 0.5 KB | per-warp | shared |
| `smem_O[16][64]` | 4 KB | — | v1 only |
| `row_max/sum[16]` | 128 B | — | yes |
| `correction[16]` | 64 B | — | v2 only |

## 5 Experimental Setup

### 5.1 Datasets

We evaluate on two standard citation network benchmarks from the Planetoid collection [8], exported to CSR format via PyTorch Geometric:

**Table 2: Dataset statistics and block-sparse format properties.**

| Dataset | Nodes | Edges | Row Windows | Avg. Deg. |
|---|---|---|---|---|
| Cora | 2,708 | 10,556 | 170 | 3.90 |
| Citeseer | 3,327 | 9,104 | 208 | 2.74 |

### 5.2 Hardware and Software

All experiments are conducted on an NVIDIA GeForce RTX 4050 GPU (Ada Lovelace architecture, compute capability sm_89, 20 SMs, 6 GB GDDR6X). Kernels are compiled with CUDA 12.x using `-O3 --use_fast_math` and target `sm_89`. Feature dimension is fixed at $d = 64$.

### 5.3 Baselines

We implement two reference baselines for correctness verification:

- **CPU Baseline:** Naive per-node sparse attention on CSR format. For each node $i$, computes dot products with all neighbors, applies numerically stable softmax, and aggregates weighted features. Serves as the ground truth.
- **GPU Baseline:** Naive GPU kernel assigning one thread per node. Uses device-side `malloc` for per-thread attention score storage. No Tensor Core usage.

### 5.4 Verification Methodology

Each test binary: (1) loads the CSR graph and converts to block-sparse format, (2) generates random Q, K, V matrices ($\mathcal{U}(-0.5, 0.5)$, seed 42), (3) computes CPU ground truth using the *exact same block-sparse semantics*, (4) launches the fused kernel, and (5) compares GPU output against CPU with element-wise tolerance $\epsilon = 0.05$ (accounting for FP16 $\rightarrow$ FP32 precision loss in WMMA).

## 6 Results

### 6.1 Kernel Execution Time

**Table 3: Kernel-only execution time on Cora ($d = 64$, 170 Row Windows) and Citeseer ($d = 64$, 208 Row Windows), measured via NVIDIA Nsight Compute (`gpu__time_duration.sum`). Times exclude launch overhead and `float2half` conversion kernels.**

| Kernel | Cora ($\mu$s) | Citeseer ($\mu$s) | Softmax | Output | Correct |
|---|---|---|---|---|---|
| `v0` | 13.25 | 14.24 | Per-block | Global mem | No |
| `v1` | 20.64 | 22.91 | Online | Shared mem | Yes |
| `v2` | **8.77** | **9.31** | Online | Registers | Yes |

Table 3 reports kernel-only execution times measured by Nsight Compute, isolating the `f3s_1tb1rw_kernel` invocation from launch overhead and auxiliary `float2half` conversion kernels. Across both datasets, `v2` achieves the fastest execution, completing in 8.77 $\mu$s on Cora and 9.31 $\mu$s on Citeseer. This represents up to a 1.53× speedup over `v0` and 2.46× over `v1`. The register-resident output optimization in `v2` eliminates the shared memory bottleneck of `v1` and also outperforms `v0` by avoiding per-warp redundant computation across column blocks.

### 6.2 Correctness Verification

All three fused kernel versions pass correctness verification against CPU ground truth on both Cora and Citeseer datasets, with zero mismatches at tolerance $\epsilon = 0.05$. Note that `v0`'s "correctness" is verified against a CPU reference that uses the same per-block softmax semantics—it would produce different results from a globally correct softmax when rows span multiple column blocks.

### 6.3 Profiling Analysis

We profile all kernel versions using NVIDIA Nsight Compute (`ncu --set full`) on the Cora and Citeseer datasets.

Figures 3 and 4 present the floating-point roofline analysis, which is the primary indicator of kernel efficiency. The roofline measures

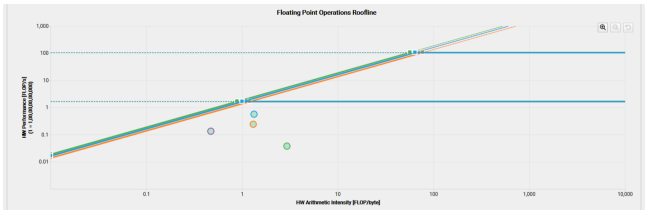


**Figure 3: Floating-point roofline analysis on Cora. Square markers represent Tensor Core (WMMA) operations at high arithmetic intensity (∼50–100 FLOP/byte), while circle markers show scalar operations in the memory-bound region (∼1–3 FLOP/byte). v2's Tensor Core operations sit closest to the compute ceiling, confirming the highest hardware utilization.**

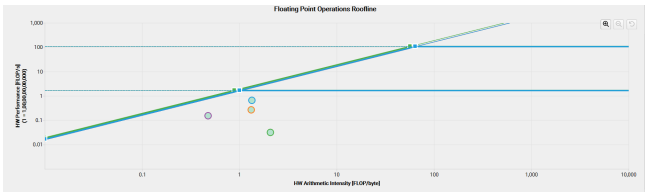


**Figure 4: Floating-point roofline analysis on Citeseer. Similar to Cora, v2 achieves the highest hardware utilization.**

how effectively each kernel utilizes hardware *per cycle of execution*, independent of total runtime. The Tensor Core operations (square markers) for all fused kernels appear at high arithmetic intensity (∼50–100 FLOP/byte), approaching the compute ceiling. Among them, v2 achieves the highest SM throughput at 21.69% of peak on Cora and 24.87% on Citeseer (vs. 14.15% and 16.05% for v0), consistent with its register-resident design eliminating shared memory bottlenecks. The scalar operations (circle markers) remain in the memory-bound region at low arithmetic intensity (∼1–3 FLOP/byte), reflecting the softmax and data-movement phases.

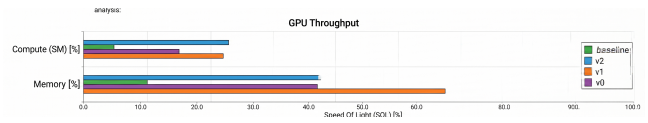


**Figure 5: GPU throughput comparison (Speed of Light %) across kernel versions on Cora. v1 and v2 achieve higher SM compute utilization than v0, while v1 shows the highest memory throughput due to its shared-memory output rescaling traffic.**

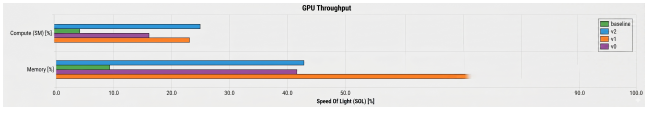


**Figure 6: GPU throughput comparison (Speed of Light %) across kernel versions on Citeseer.**

Figures 5 and 6 show GPU throughput utilization. The fused kernels achieve significantly higher SM compute utilization (25–28% on Cora, 13–25% on Citeseer) compared to the naive baseline. Notably, v1 exhibits the highest memory throughput (61.48% on Cora, 67.90% on Citeseer) among the fused kernels—this is not a sign of efficiency but rather reflects the per-iteration shared memory round-trips for `smem_O[16][64]` rescaling. v2 reduces memory throughput to 37.19% (Cora) and 42.80% (Citeseer) by keeping the output accumulator in registers, converting that memory traffic into useful compute.

# 7 Discussion

## 7.1 v1 Slowdown Analysis

Across both datasets, v1 is slower than v0 (1.56× on Cora, 1.61× on Citeseer) and significantly slower than v2 (2.35× on Cora, 2.46× on Citeseer), attributable to three factors:

(1) **Shared memory output rescaling:** Each iteration, the correction factor $e^{m_{\text{old}}-m_{\text{new}}}$ must be applied to all 1,024 elements of `smem_O[16][64]`. With only 16 threads performing this (warp 0, lanes 0–15), this serialized loop over 64 feature dimensions per row becomes a bottleneck.
(2) **Extra barrier:** After SpMM writes to `smem_O`, a `__syncthreads()` is required before the next iteration can rescale the same buffer. This adds latency per iteration.
(3) **Sequential column-block processing:** Unlike v0 which distributes column blocks across warps in parallel, v1 must process them sequentially to maintain coherent online softmax state. This serializes the critical path, particularly on graphs where rows span few column blocks.

## 7.2 v2 Optimization Analysis

Version 2 achieves the fastest kernel execution (8.77 $\mu$s on Cora, 9.31 $\mu$s on Citeseer) through two key optimizations:

(1) **Register-resident accumulator:** By keeping `frag_O` in WMMA register fragments, the 4 KB shared memory allocation for `smem_O` is eliminated, and rescaling operates on 8 register elements per thread rather than 64 shared memory locations per row.
(2) **Barrier elimination:** Since SpMM writes only to registers (private to each warp), the post-SpMM `__syncthreads()` is removed, allowing the next iteration's SDDMM phase to begin immediately.

The fragment layout mapping (groupID = lane_id / 4, with elements [0,1,4,5] mapping to row groupID and [2,3,6,7] to row groupID + 8) enables correct per-row scaling without an intermediate shared memory step. The 1.51–1.53× speedup over v0 (despite v0 parallelizing column blocks across warps) demonstrates that the register-resident approach more than compensates for the sequential column-block iteration required by online softmax. The roofline analysis (Figures 3 and 4) corroborates this: v2 achieves the highest SM throughput, confirming that it extracts more useful compute per cycle.

## 7.3 Block-Sparse Overhead

The block-sparse format introduces padding overhead: rows within a 16×16 block that have no actual edges still participate in WMMA operations with zero-valued Q/K/V entries. For sparse graphs like Cora (average degree 3.9), the number of active 16×16 blocks can significantly exceed the actual edge count. However, this overhead is more than compensated by the ability to use Tensor Cores, which provide ∼16× throughput over scalar FP32 operations.

## 7.4 Limitations

- **Fixed tile size:** The 16×16 WMMA tile is hardcoded. Graphs with very high or very low average degree may benefit from different tile granularities.
- **Feature dimension:** The kernel assumes $d = 64$. Supporting variable feature dimensions requires compile-time or runtime adaptation of the number of feature chunks.
- **sm_80+ dependency:** The fragment rescaling in `v2` relies on the WMMA accumulator fragment layout specific to sm_80 and later architectures.
- **Scale factor:** The attention scores are not divided by $\sqrt{d}$ as in standard scaled dot-product attention [9]. This is straightforward to add but was omitted for simplicity.

# 8 Related Work

**Fused attention kernels.** FlashAttention [1] pioneered the fusion of attention computation for dense transformers, using tiling and online softmax to avoid materializing the $N \times N$ attention matrix. Our work applies similar principles to the sparse setting, where the attention pattern is determined by graph adjacency rather than being dense.

**Sparse GNN kernels.** FeatGraph [2] and GE-SpMM [3] optimize individual SpMM or SDDMM kernels for GNNs but do not fuse across stages. SparseTIR [11] provides composable abstractions for sparse computation but relies on code generation rather than hand-tuned kernels.

**Fused3S.** Our implementation is based on the approach proposed by Li and Chandramowlishwaran [5], who introduced the fused SDDMM + Softmax + SpMM pipeline on Tensor Cores. We implement their core ideas from scratch and add progressive optimization stages (`v0` → `v1` → `v2`) to systematically study the impact of each optimization.

# 9 Conclusion

We implemented Fused3S, a CUDA kernel that fuses the three-stage sparse attention pipeline (SDDMM + Softmax + SpMM) into a single GPU kernel using NVIDIA Tensor Cores. Through three progressive kernel versions, we demonstrated that:

(1) **Fusion eliminates memory traffic:** By keeping intermediate attention scores and weights in shared memory and registers, the fused kernel avoids the global memory round-trips of the standard three-kernel pipeline.
(2) **Online softmax enables correctness:** The streaming softmax algorithm produces globally correct attention weights in a single pass over column blocks, unlike the per-block approximation in `v0`.
(3) **Register-resident output is fastest:** Moving the output accumulator from shared memory (`v1`) to WMMA register fragments (`v2`) achieves the fastest kernel execution at 8.77 $\mu$s on Cora and 9.31 $\mu$s on Citeseer—up to a 1.53× speedup over `v0` and 2.46× over `v1`—while maintaining globally correct softmax.

The roofline analysis confirms that `v2` achieves the highest hardware utilization among all versions, with Tensor Core operations approaching the compute ceiling at high arithmetic intensity. Future work includes supporting variable feature dimensions, exploring larger tile sizes on newer architectures (e.g., Hopper's wgmma), and integrating Fused3S into a full GNN training pipeline.


# Acknowledgments

We thank Prof. Sameer Kulkarni for his guidance, feedback, and support as the project advisor for CS299 (Independent Study) at IIT Gandhinagar.


# References


[1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. https://arxiv.org/abs/2205.14135

[2] Yuwei Hu, Zidong Du, Qi Guo, Ling Han, and Yunji Chen. 2020. FeatGraph: A Flexible and Efficient Backend for Graph Neural Network Systems. In *SC '20: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. doi:10.1109/SC41405.2020.00075

[3] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. 2021. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. In *SC '21: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. doi:10.1145/3458817.3476205

[4] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1609.02907

[5] Zhaodong Li and Aparna Chandramowlishwaran. 2025. Fused3S: Fast Sparse Attention on Tensor Cores. *arXiv preprint arXiv:2505.08098* (2025). doi:10.48550/arXiv.2505.08098

[6] Maxim Milakov and Natalia Gimelshein. 2018. Online Normalizer Calculation for Softmax. *arXiv preprint arXiv:1805.02867* (2018). https://arxiv.org/abs/1805.02867

[7] NVIDIA Corporation. 2024. *CUDA C++ Programming Guide: Warp Matrix Functions*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma Accessed: 2026-04-20.

[8] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–106. doi:10.1609/aimag.v29i3.2157

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. https://arxiv.org/abs/1706.03762

[10] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1710.10903

[11] Zihao Ye, Hongyi Zhang, Yinan Chen, Lianmin Zheng, and Tianqi Chen. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. doi:10.1145/3575693.3575700